3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

Classes : Offline & Online

Daily Teaching : 2 Hours

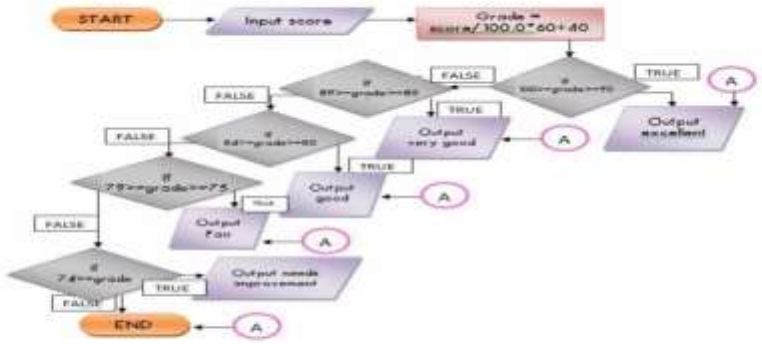
Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 25 – Module Assessment



FOUNDATION

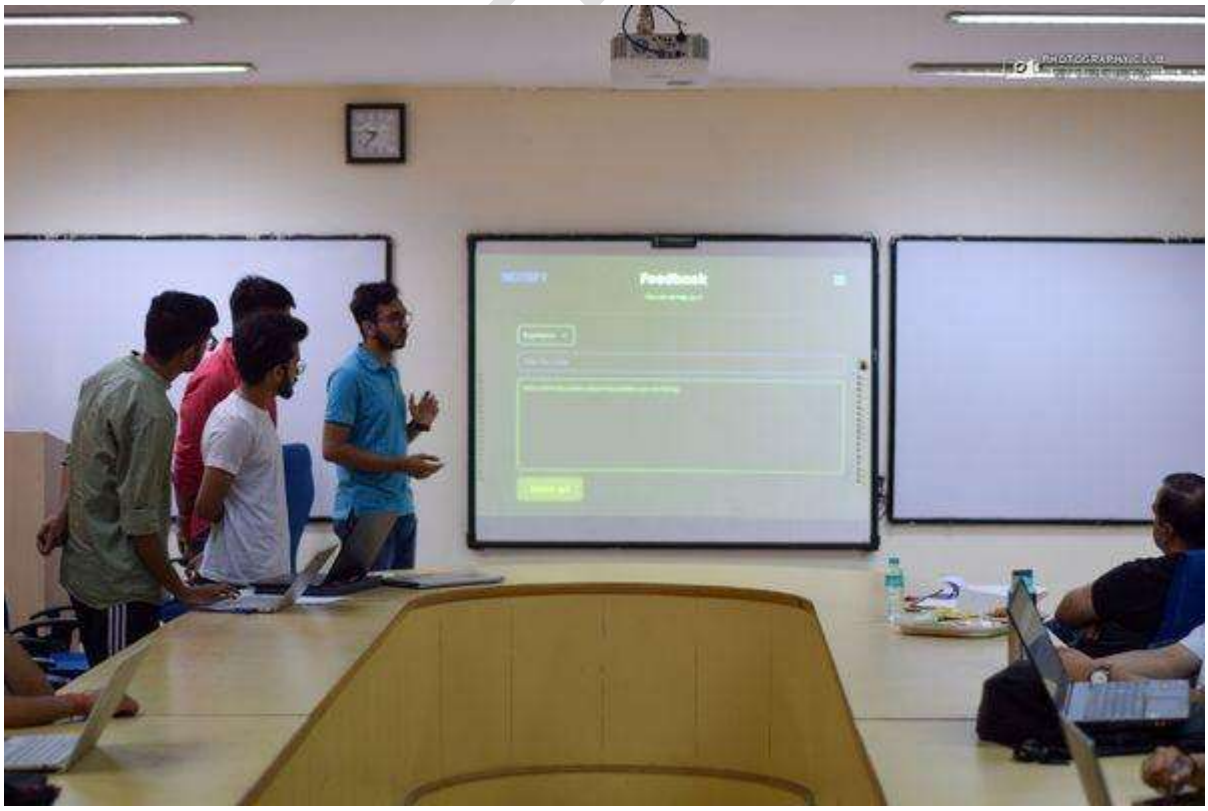


PSEUDOCODE:

```
score = input "What is your score?"
grade =score/100*60+40
if 100>=grade>= 90:
        output excellent
elif 89>=grade>= 85:
        output very good
elif 84>=grade>= 80:
        output good
elif 79>=grade>= 75:
        output fair
else:
        output needs improvement
```

PYTHON CODE:

```
score=input("What is your score?")
grade=(score/100*60+40)
if 100 >= grade >= 90:
    print("Excellent")
elif 89 >= grade >= 85:
    print("Very Good")
elif 84 >= grade >= 80:
    print("Good")
elif 79 >= grade >= 75:
    print("Fair")
else:
    print("Needs Improvement")
```

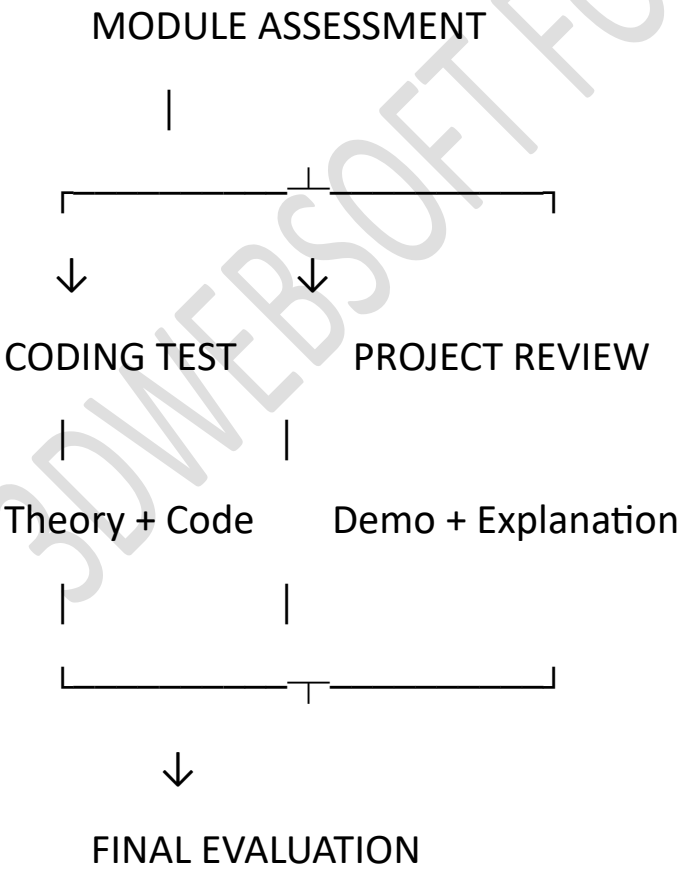


1. Introduction

A **Module Assessment** is conducted after completing the Advanced Python module to check whether students can:

- Understand Python concepts.
- Write Python programs independently.
- Find and correct errors.
- Apply concepts to real-world problems.
- Build a small application.
- Explain their project confidently.
- Demonstrate programming logic.

The assessment has **two major parts**:



3DWEBSOFT FOUNDATION

2. Learning Objectives

By the end of this assessment, students should be able to:

1. Write Python programs without copying code.
2. Use variables, data types and operators.
3. Apply conditions and loops.
4. Create and use functions.
5. Work with flexible functions.
6. Use lambda functions.
7. Process data using map(), filter() and reduce().
8. Work with modules and packages.
9. Read and write files.
10. Handle CSV and JSON data.
11. Handle exceptions.
12. Debug Python programs.
13. Create classes and objects.
14. Apply OOP concepts.
15. Use Python libraries.
16. Build and explain a mini project.

3DWEBSOFT FOUNDATION

3. Assessment Structure

A suggested assessment pattern is:

Component	Marks
Coding Test	40
Project Implementation	30
Project Demonstration	15
Viva / Explanation	10
Code Quality	5
Total	**100**

Suggested Passing Score

50/100

The trainer can change the passing score according to the institution's assessment policy.

4. Part A – Coding Test

The coding test checks individual programming ability.

Recommended Duration

60–90 minutes

Coding Test Pattern

Coding Test

```
|
├── Basic Programming
├── Functions
├── Data Processing
```

3DWEBSOFT FOUNDATION

├── File Handling

├── Exception Handling

├── OOP

└── Debugging

5. Section A – Basic Python Questions

Question 1

Write a Python program to check whether a number is even or odd.

```
number = int(input("Enter a number: "))


if number % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

Sample Output

```
Enter a number: 24
Even number
```

6. Section B – Loops

Question 2

Write a program to print numbers from 1 to 10.

```
for i in range(1, 11):
```

3DWEBSOFT FOUNDATION

```
    print(i)
```

Challenge

Print only even numbers.

```
for i in range(1, 11):
    if i % 2 == 0:
        print(i)
```

7. Section C – Functions

Question 3

Create a function that calculates the square of a number.

```
def square(number):
    return number * number
num = int(input("Enter number: "))
print("Square:", square(num))
```

Expected Output

Enter number: 8

Square: 64

*8. Section D – *args*

Question 4

Create a function that accepts any number of numbers and returns their sum.

```
def total(*args):
```

3DWEBSOFT FOUNDATION

```
    return sum(args)
print(total(10, 20, 30))
print(total(5, 10, 15, 20))
```

Output:

60

50

9. Section E – Lambda

Question 5

Use a lambda function to find the square of a number.

```
square = lambda x: x * x

print(square(7))
```

Output:

49

10. Section F – map() and filter()

Question 6

Use map() to double every number.

```
numbers = [1, 2, 3, 4, 5]

result = list(map(lambda x: x * 2, numbers))
```

3DWEBSOFT FOUNDATION

```
print(result)
```

Output:

```
[2, 4, 6, 8, 10]
```

Question 7

Use filter() to find numbers greater than 50.

```
marks = [45, 67, 32, 89, 76]


result = list(filter(lambda x: x > 50, marks))
print(result)
```

Output:

```
[67, 89, 76]
```

11. Section G – File Handling

Question 8

Write student names into a text file.

```
names = ["Aisha", "Rahul", "Priya"]
with open("students.txt", "w") as file:
    for name in names:
        file.write(name + "\n")


print("Data saved!")
```

Expected Output

```
Data saved!
```

SOWEBSOFT FOUNDATION

The file contains:

Aisha

Rahul

Priya

12. Section H – Exception Handling

Question 9

Write a program that safely accepts two numbers and divides them.

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    result = a / b
    print("Result:", result)
except ValueError:
    print("Please enter numbers only.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Learning Point

Students should understand:

try

↓

Run risky code

↓

SUWEBSOFT FOUNDATION

Error?

↓

except

↓

Handle error safely

13. Section I – Debugging

Students may receive an incorrect program and must identify the error.

Question 10

Find and correct the error:

```
numbers = [10, 20, 30]


for number in numbers
    print(number)
```

Correct Answer

```
numbers = [10, 20, 30]


for number in numbers:
    print(number)
```

Error

A colon : was missing after the for statement.

3DWEBSOFT FOUNDATION

14. Section J – OOP

Question 11

Create a Student class with a name and marks.

```
class Student:


    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)
student1 = Student("Aisha", 88)
student1.display()
```

Output:

Name: Aisha

Marks: 88

SDWEBSOFT FOUNDATION

15. Coding Test – Practical Challenge

Give students one larger problem.

Problem: Student Result Calculator

Create a program that:

1. Accepts student name.
2. Accepts three subject marks.

3. Calculates total.
4. Calculates percentage.
5. Displays grade.
6. Handles invalid marks.

Expected Logic

Student Name

↓

Enter 3 Marks

↓

Validate Marks

↓

Calculate Total

↓

Calculate Percentage

↓

Calculate Grade

↓

Display Result

3DWEBSOFT FOUNDATION

16. Part B – Project Review

Students should present the mini project created during Class 24.

Examples:

- Student Management System

- Expense Tracker
- Library Management System
- Quiz Application
- Attendance System
- To-Do List
- Student Performance Analyzer





17. Project Review Process

The trainer follows this sequence:

Project Submission

↓

Code Inspection

↓

Project Execution

↓

Feature Demonstration

↓

Code Explanation

↓

Viva Questions

↓

Feedback

↓

Marks

3DWEBSOFT FOUNDATION

18. Project Review – Step 1: Problem Statement

Students must explain:

What problem does your project solve?

Example:

“My project is a Student Management System. It helps teachers store student information, calculate results and search student records.”

19. Project Review – Step 2: Features

Students should list the features.

Example:

Student Management System

```
|
 ├── Add Student
 ├── View Students
 ├── Search Student
 ├── Calculate Result
 ├── Find Top Student
 ├── Save Data
 └── Exit
```

20. Project Review – Step 3: Python Concepts

Students should identify the concepts used.

Example:

Concept	**Project Usage**
Variables	Store values
Lists	Store multiple records

3DWEBSOFT FOUNDATION

Concept	Project Usage
Dictionaries	Store student details
Functions	Separate operations
Loops	Display/search records
Conditions	Grade calculation
Exception Handling	Validate input
CSV	Save data
OOP	Optional advanced structure

21. Project Review – Step 4: Live Demonstration

Students should run the application in front of the trainer.

Demonstration Checklist

☐ Program starts correctly

☐ Menu works

☐ Data can be entered

☐ Data is displayed

☐ Search works

☐ Calculations are correct

☐ Invalid input is handled

☐ Data is saved

☐ Program exits correctly

3DWEBSOFT FOUNDATION

22. Project Review – Step 5: Code Explanation

Students should explain important sections of their code.

For example:

```
def calculate_percentage(student):
    total = (
        student["python"]
        + student["ai"]
        + student["cyber"]
    )

    return total / 3
```

The student should explain:

“This function receives a student dictionary, adds the three subject marks and returns the average percentage.”

The goal is not only to **write code**, but to **understand the code**.

23. Project Viva Questions

Basic Questions

1. What is the name of your project?

2. What problem does it solve?

3. Why did you choose this project?

4. Which programming language did you use?

5. Which Python concepts did you use?

SUNEBSOFT FOUNDATION

Technical Questions

6. Why did you use functions?

7. Why did you use a list?

8. Why did you use a dictionary?

9. What is exception handling?

10. What happens if the user enters invalid data?

11. Why did you use CSV/JSON?

12. What is the purpose of return?

13. What is the difference between a list and dictionary?

14. What is a class?

15. What is an object?

24. Higher-Level Viva Questions

For stronger 11th–12th standard students:

Q1. How can you improve your project?

Possible answer:

"I can add a login system, database storage, graphical user interface and data visualization."

Q2. How would you protect user data?

Possible answer:

"I would use authentication, proper validation and secure storage."

Q3. How would you handle a large number of students?

Possible answer:

3DWEBSOFT FOUNDATION

“Instead of storing everything only in memory, I could use a database such as SQLite or another database system.”

Q4. How can you make your program easier to maintain?

Possible answer:

“I can divide the program into functions or classes, use meaningful names and add comments where necessary.”

25. Project Evaluation Rubric

Evaluation Area	**Marks**
Problem Definition	5
Program Functionality	10
Python Concepts	5
File/Data Handling	5
Error Handling	5
User Interface/Menu	5
Project Demonstration	10
Code Explanation	5
Viva	10
Total	**60**

The trainer can scale these marks to match the overall **100-mark assessment**.

3DWEBSOFT FOUNDATION

26. Coding Test Evaluation Rubric

Area	Marks
Basic Python	5
Conditions & Loops	5
Functions	5
Data Processing	5
File Handling	5
Exception Handling	5
OOP	5
Debugging	5
Total	**40**

27. Performance Levels

Score	Performance
90–100	Excellent
80–89	Very Good
70–79	Good
60–69	Satisfactory
50–59	Needs Improvement
Below 50	Requires Additional Practice

3DWEBSOFT FOUNDATION

28. Practical Assessment Activity

Challenge: Build a Result Analyzer

Students receive:

```
students = [
    {"name": "Aisha", "marks": 90},
    {"name": "Rahul", "marks": 76},
    {"name": "Priya", "marks": 88},
    {"name": "Arjun", "marks": 65}
]
```

They must create a program that:

1. Displays all students.
2. Calculates average marks.
3. Finds the highest scorer.
4. Finds students above 80.
5. Assigns grades.
6. Handles an empty list.

Expected Processing

Student Data

↓

Read Records

↓

Calculate Average

↓

9JWEBSOFT FOUNDATION

Find Highest

↓

Filter > 80

↓

Assign Grades

↓

Display Report

29. Student Self-Assessment

Students can complete this checklist after the assessment.

Skill	Yes	Need Practice
I can write Python programs independently	☐	☐
I can create functions	☐	☐
I understand lambda functions	☐	☐
I can use map() and filter()	☐	☐
I can work with files	☐	☐
I can work with CSV/JSON	☐	☐
I can handle exceptions	☐	☐
I can debug errors	☐	☐
I understand OOP	☐	☐

3DWEBSOFT FOUNDATION

Skill	Yes Need Practice
I can explain my project	☐ ☐

30. Trainer Feedback Format

After the assessment, the trainer can provide feedback:

Student Name:

Strengths:

- _____
- _____
- _____

Areas for Improvement:

- _____
- _____
- _____

Recommended Practice:

- _____
- _____

Final Score:

____ / 100

31. Assessment Rules for Students

Students should follow these rules:

3DWEBSOFT FOUNDATION

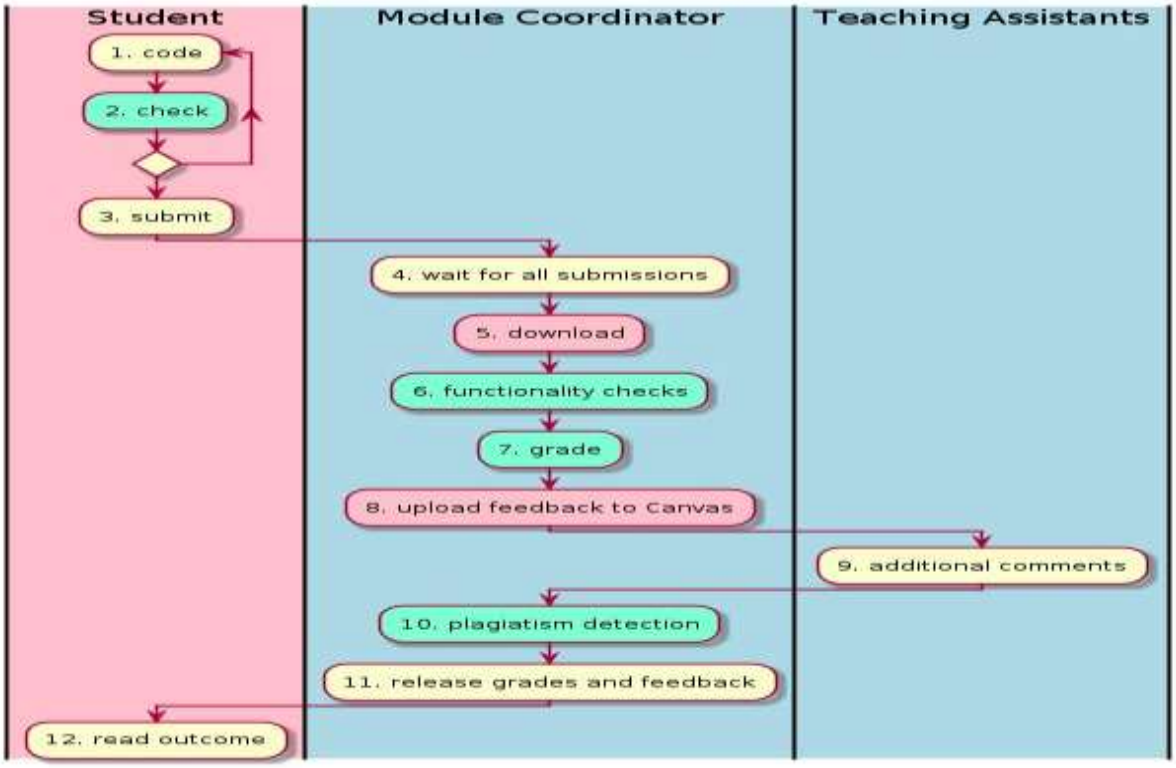
1. Read every question carefully.
2. Plan the solution before coding.
3. Use meaningful variable names.
4. Maintain proper indentation.
5. Test the program before submission.
6. Do not copy another student's code.
7. Handle errors properly.
8. Explain your own project.
9. Keep the code readable.
10. Submit the final working version.

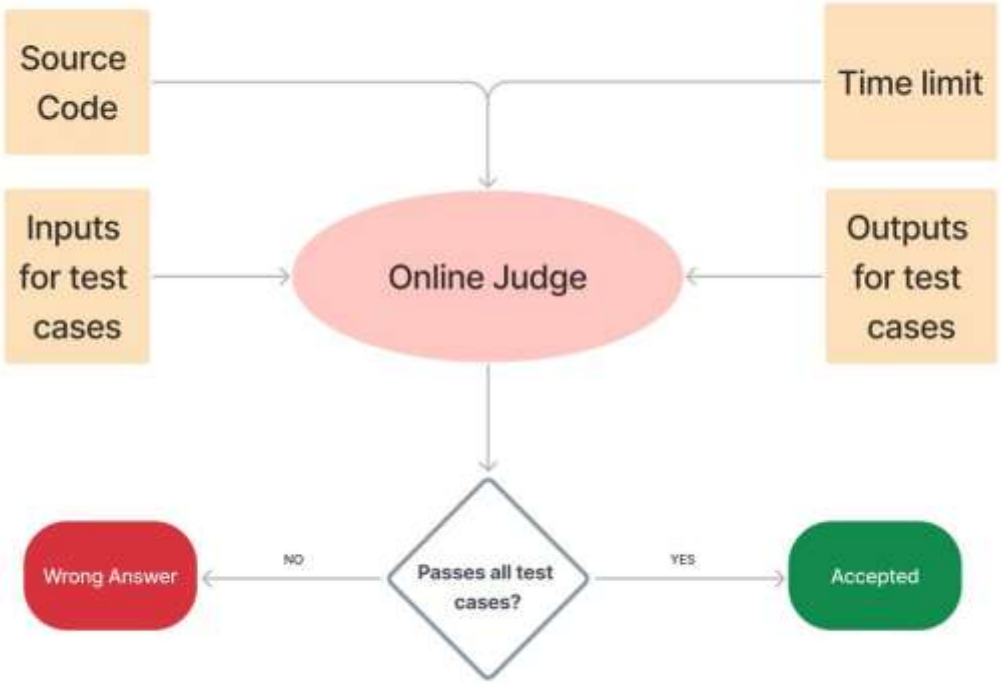
32. Important Difference: Coding Test vs Project Review

Coding Test	Project Review
Individual	Individual/team depending on project
Time-limited	Demonstration-based
Tests coding skills	Tests application skills
Short programs	Complete application
Problem-solving	Real-world implementation
Written/practical questions	Demo + Viva

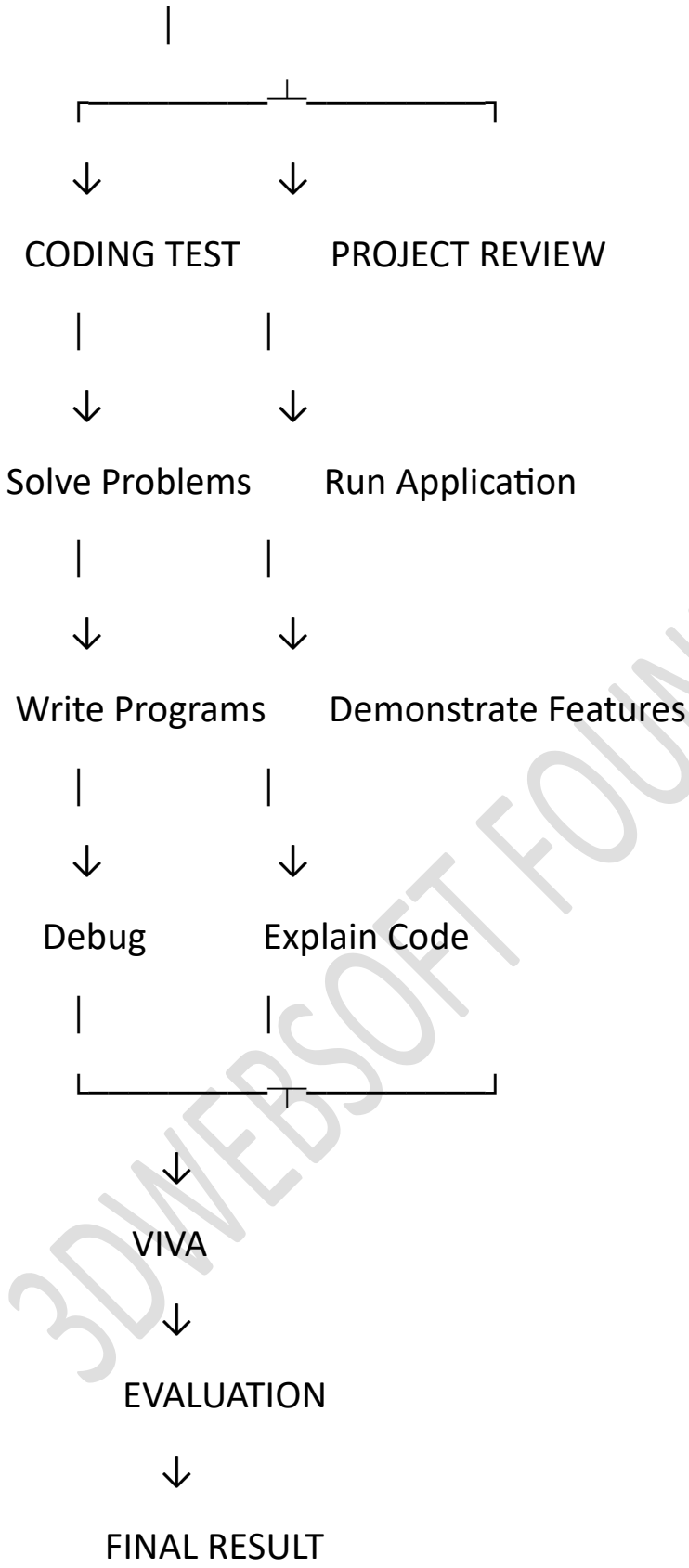
33. Final Assessment Flow

3DWEBSOFT FOUNDATION





MODULE ASSESSMENT



3DWEBSOFT FOUNDATION

34. Quick Revision

Students should remember:

Python knowledge + Problem Solving + Coding + Testing + Project Explanation = Practical Programming Skill

The assessment should not only ask:

“Can you write Python code?”

It should also check:

“Can you use Python to solve a problem and explain your solution?”

35. Final Takeaway

By the end of **Class 25 – Module Assessment**, students should demonstrate that they can move from **individual Python concepts to independent application development**.

Learn Python

↓

Practice Concepts

↓

Solve Coding Problems

↓

Build Mini Project

↓

Test & Debug

↓

Present Project

3DWEBSOFT FOUNDATION

↓

Answer Viva

↓

Ready for Next Module

3DWEBSOFT FOUNDATION